

SMART POWER MANAGEMENT ADAPTER

FINAL PROJECT REPORT

PROJECT TITLE

Smart Power Management Adapter with Real-Time Equipment Monitoring and Intelligent Control Using Object-Oriented Programming in C++

ABSTRACT

The Smart Power Management Adapter is a console-based simulation system developed using C++ and Object-Oriented Programming (OOP) principles to mimic the behavior of an actual IoT-based smart energy monitoring and control system.

The project was designed to simulate a real embedded smart power management environment without requiring physical hardware components such as ESP32 microcontrollers, relay modules, current sensors, or voltage sensors. The system continuously monitors simulated electrical appliances, dynamically calculates power consumption, accumulates energy usage, performs intelligent overload management, generates monthly reports, stores analytics using binary files, and simulates cloud synchronization.

The software implements real-time dashboard updates similar to industrial SCADA systems and smart home automation platforms. The system architecture follows layered IoT principles consisting of the User Layer, Communication Layer, Processing Layer, Sensor/Actuator Layer, and Data Storage Layer.

The project demonstrates practical implementation of major OOP concepts including abstraction, encapsulation, inheritance, polymorphism, and composition. The developed system provides a professional simulation of modern smart energy management systems while remaining fully executable as a console application.

1. INTRODUCTION

Energy consumption management has become increasingly important due to the growing demand for electricity and the need for efficient utilization of power resources. Traditional electrical systems provide limited visibility into appliance-level power consumption, making it difficult for users to monitor and optimize energy usage.

Modern smart energy systems solve this challenge by integrating sensing, processing, communication, and intelligent control mechanisms into a unified platform.

This project presents a Smart Power Management Adapter developed as a software simulation using C++. The system mimics a real IoT-enabled energy management platform capable of monitoring multiple appliances in real time.

The project was implemented as a console application because hardware components were unavailable during development. Despite this limitation, the system successfully simulates real embedded-system behavior.

2. PROBLEM STATEMENT

Traditional power systems lack:

- Real-time appliance-level monitoring
- Intelligent overload protection
- Remote appliance control
- Dynamic analytics and reporting
- Smart energy management

Additionally, many students and developers may not have access to physical IoT hardware during software development and testing.

This project addresses these challenges by providing a complete software simulation of a smart energy management system.

3. OBJECTIVES

3.1 Main Objective

To design and implement a real-time Smart Power Management Adapter capable of simulating intelligent energy monitoring and appliance control using C++.

3.2 Specific Objectives

The project aims to:

- Simulate current and voltage sensor readings
 - Dynamically calculate power and energy
 - Monitor appliances in real time
 - Allow appliance ON/OFF control
 - Generate monthly energy reports
 - Store analytics using binary files
 - Simulate cloud synchronization
 - Implement intelligent overload protection
 - Demonstrate Object-Oriented Programming concepts
-

4. SYSTEM ARCHITECTURE

The system is organized into multiple logical layers.

4.1 User Layer

This layer provides user interaction interfaces.

Components include:

- Real-Time Monitoring Dashboard
- Equipment Control Center
- Diagnostics Center
- Scheduling Center
- Report Management Interface

Responsibilities:

- Display live appliance data
 - Accept user commands
 - Display system status
 - Navigate system operations
-

4.2 Communication Layer

This layer simulates communication between devices and cloud services.

Responsibilities:

- Cloud synchronization

- Data transmission
 - Log management
 - Simulated communication protocols
-

4.3 Processing Layer

This layer acts as the brain of the system.

Implemented using:

SmartEnergyController class

Responsibilities:

- Sensor data processing
 - Power calculations
 - Energy accumulation
 - Intelligent load management
 - Dashboard operation
 - Command execution
-

4.4 Sensor and Actuator Layer

This layer simulates hardware interaction.

Sensors

- CurrentSensor
- VoltageSensor

Actuators

- Relay modules

Responsibilities:

- Simulate electrical measurements
 - Simulate appliance switching
 - Mimic real hardware behavior
-

4.5 Data Storage Layer

Responsible for persistent data management.

Generated files:

- analytics.bin
 - monthly_report.txt
 - cloud_log.txt
-

5. FUNCTIONAL REQUIREMENTS

The system shall:

- Continuously monitor appliances
 - Display live power consumption
 - Allow appliance control
 - Generate reports
 - Store analytics
 - Simulate cloud synchronization
 - Detect overload conditions
 - Automatically disconnect low-priority appliances
-

6. NON-FUNCTIONAL REQUIREMENTS

Performance

- Fast refresh rate
- Real-time monitoring
- Efficient execution

Reliability

- Stable operation
- Persistent storage
- Continuous simulation

Maintainability

- Modular architecture
- Well-commented code
- OOP implementation

Portability

- Runs on CodeBlocks
 - Compatible with Windows and Linux
-

7. OBJECT-ORIENTED PROGRAMMING CONCEPTS

7.1 Encapsulation

Class data is protected using private and protected access modifiers.

Examples:

- Relay states
 - Appliance data
 - Sensor readings
-

7.2 Inheritance

Derived appliance classes inherit from the Appliance base class.

Derived classes include:

- Fan
 - Refrigerator
 - Television
 - Heater
 - WaterPump
-

7.3 Polymorphism

Implemented using virtual functions.

Example:

```
virtual float readValue() = 0;
```

7.4 Abstraction

Implemented through abstract Sensor classes.

7.5 Composition

Appliance objects contain:

- CurrentSensor

- VoltageSensor
 - Relay
-

8. SYSTEM MODULES

8.1 Sensor Module

Simulates:

- Current readings
 - Voltage fluctuations
-

8.2 Appliance Module

Manages:

- Appliance behavior
 - Power calculation
 - Energy accumulation
-

8.3 Controller Module

Handles:

- Real-time simulation
 - Dashboard management
 - System processing
 - Load shedding
-

8.4 Analytics Module

Handles:

- Binary analytics storage
 - Analytics retrieval
 - Cloud logging
-

8.5 Reporting Module

Responsible for:

- Monthly report generation
 - Usage summaries
 - Appliance analytics
-

9. SYSTEM IMPLEMENTATION

The system was implemented using standard C++.

Major implementation features include:

- Dynamic real-time simulation loop
- Appliance state management
- Embedded-system style operation
- Continuous dashboard refresh
- Binary file storage
- Intelligent overload protection

The application mimics real ESP32 firmware execution using continuous loops and sensor polling.

10. CONSOLE OUTPUT DEMONSTRATION

10.1 Main Menu Output

```
===== SMART  
ENERGY MANAGEMENT SYSTEM  
=====
```

[1] Live Energy Monitoring Dashboard [2] Generate Monthly Energy Report [3] Save Analytics To Binary Storage [4] View Stored Analytics [5] System Fault Diagnostics [6] Appliance Scheduling Center [0] Shutdown System

```
===== Select  
System Operation:
```

10.2 Real-Time Dashboard Output

```
===== REAL-  
TIME ENERGY MONITORING DASHBOARD  
=====
```


Appliance Voltage Current Power(W) Energy(kWh) Status

Fan 228.3 0.4 91.3 0.012 ON Refrigerator 231.0 1.5 346.5 0.056 ON Television 229.1
0.6 137.4 0.023 ON Heater 0.0 0.0 0.0 0.000 OFF Water Pump 230.2 5.4 1243.1 0.182
ON _____ TOTAL POWER CONSUMPTION :
1818.3 W MAX SYSTEM LIMIT : 2500 W SYSTEM STATUS : ACTIVE CLOUD
CONNECTIVITY : CONNECTED OVERLOAD PROTECTION : ENABLED _____

10.3 Equipment Control Center Output

===== EQUIPMENT
CONTROL CENTER =====
[1] Fan STATUS: ON [2] Refrigerator STATUS: ON [3] Television STATUS: ON [4]
Heater STATUS: OFF [5] Water Pump STATUS: ON
===== [6] Turn ON
Equipment [7] Turn OFF Equipment [8] Return To Dashboard
=====

10.4 Diagnostics Output

===== SYSTEM
DIAGNOSTICS CENTER
=====
Checking current sensors..... OK Checking voltage sensors..... OK
Checking relay modules..... OK Checking appliance communication..... OK
Checking cloud synchronization..... OK Checking binary storage..... OK
Checking overload protection..... OK
System Status: FULLY OPERATIONAL

10.5 Overload Detection Output

WARNING: SYSTEM OVERLOAD DETECTED Disconnected: Television

10.6 Analytics Output

===== STORED ANALYTICS
=====

Appliance : Fan Power : 92.4 W Energy : 0.013 kWh

Appliance : Refrigerator Power : 347.1 W Energy : 0.057 kWh

11. DATA PROCESSING AND CALCULATIONS

Power Calculation

The system calculates power using:

$$P = V \times I$$

Where:

- P = Power in Watts
- V = Voltage in Volts
- I = Current in Amperes

Example:

If:

Voltage = 230V Current = 1.5A

Then:

$$P = 230 \times 1.5 \quad P = 345W$$

Energy Calculation

Energy is accumulated over time.

Formula:

$$E = (P / 1000) \times \text{Time}$$

Example:

If:

Power = 345W Time = 1 hour

Then:

$$E = (345 / 1000) \times 1 \quad E = 0.345 \text{ kWh}$$

12. FILE GENERATION

The system generates several files.

12.1 Monthly Report File

File Name:

monthly_report.txt

Contents:

- Appliance usage
 - Total energy
 - Power statistics
 - Appliance status
-

12.2 Binary Analytics File

File Name:

analytics.bin

Contents:

- Power analytics
 - Historical energy data
 - Appliance information
-

12.3 Cloud Log File

File Name:

cloud_log.txt

Contents:

- Timestamps
 - Appliance readings
 - Synchronization events
-

13. INTELLIGENT LOAD MANAGEMENT

The system includes overload protection.

Process:

1. Total power is calculated.
2. Power is compared against maximum threshold.
3. If threshold is exceeded:
 - Low-priority appliances are disconnected.
4. System stability is restored.

Benefits:

- Prevents overload
 - Simulates industrial smart-grid logic
 - Demonstrates intelligent automation
-

14. ADVANTAGES OF THE SYSTEM

- Real-time monitoring
 - Dynamic live dashboard
 - Embedded-system simulation
 - Intelligent load management
 - Persistent analytics storage
 - Professional OOP implementation
 - Hardware-independent testing
 - Expandable architecture
-

15. LIMITATIONS

- No actual hardware integration
 - No real cloud connectivity
 - Simulated sensors only
 - No graphical interface
 - No mobile application
-

16. FUTURE IMPROVEMENTS

Future enhancements may include:

- ESP32 hardware integration

- MQTT communication
 - Firebase integration
 - Mobile application development
 - Web dashboard implementation
 - AI-based energy optimization
 - Smart scheduling algorithms
 - Renewable energy integration
-

17. CONCLUSION

The Smart Power Management Adapter successfully demonstrates how intelligent energy monitoring and appliance control systems operate in modern IoT and embedded environments.

The project combines:

- Real-time monitoring
- Intelligent automation
- Persistent analytics
- Dynamic simulation
- Object-Oriented Programming
- Embedded-system principles

The developed software provides a professional simulation of industrial smart energy management systems while remaining executable as a standalone console application.

The project also demonstrates strong software engineering practices and practical implementation of advanced C++ programming concepts.
